

Task 0: Research Part

Making Big Models Smaller with Quantization

Dalah

IBT × GGateway Data Science and Machine Learning Bootcamp
Cellula Technologies

September 30, 2026

Contents

1	The Problem: Models Are Too Big	2
2	What Is Quantization?	2
2.1	The Math	2
2.2	Symmetric Quantization	3
2.3	Example by Hand	3
2.4	Why Fewer Bits Means More Error	3
3	Types of Quantization	3
3.1	Popular Methods for Large Language Models	4
4	Code Examples	4
4.1	Example 1: Quantization from Scratch	4
4.2	Example 2: Quantizing a Real BERT Model	4
4.3	Example 3: Loading LLaMA in 4-bit	5
5	Results: Memory by Precision	5
6	Advantages and Trade-offs	6
6.1	Advantages	6
6.2	Trade-offs	6
7	Other Ways to Make Models Smaller	6
8	Conclusion	6

1 The Problem: Models Are Too Big

Models like BERT and LLaMA have a lot of parameters (also called weights). A parameter is just a number the model learned during training. Normally, each number is saved in **FP32**, which uses 4 bytes (32 bits).

So the memory a model needs is:

$$\text{Memory (bytes)} = P \times \frac{b}{8} \quad (1)$$

where P is the number of parameters and b is the number of bits used for each one.

For example, LLaMA-7B has $P = 7 \times 10^9$ parameters. In FP32 ($b = 32$):

$$7 \times 10^9 \times \frac{32}{8} = 28 \times 10^9 \text{ bytes} = 28 \text{ GB} \quad (2)$$

That is too big for most normal GPUs. The table below shows the size of some models with different numbers of bits.

Model	Parameters	FP32	FP16	INT8	INT4
BERT-base	110 M	0.44 GB	0.22 GB	0.11 GB	0.06 GB
LLaMA-7B	7 B	28 GB	14 GB	7 GB	3.5 GB
LLaMA-70B	70 B	280 GB	140 GB	70 GB	35 GB

Table 1: Approximate model sizes (weights only) at different precisions.

Big models are a problem because they need:

- a lot of memory (RAM or GPU memory),
- a lot of time to load and run,
- expensive hardware, so it is hard to use them on phones or small servers.

2 What Is Quantization?

Quantization means saving the model’s numbers with fewer bits. For example, we change FP32 numbers (32 bits) into INT8 numbers (8 bits). The model becomes smaller and often faster, and we lose only a little accuracy.

A simple way to think about it: imagine a ruler with very tiny marks (FP32). Quantization is like using a ruler with bigger marks (INT8). We round each measurement to the nearest mark. It is less exact, but we need much less space to write it down.

2.1 The Math

We map a real number x to an integer q using a **scale** s and a **zero-point** z :

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right) \quad (3)$$

$$q = \text{clamp}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right) \quad (4)$$

Here $x_{\min}$ and $x_{\max}$ are the smallest and biggest real values, and $q_{\min}$ and $q_{\max}$ are the smallest and biggest integers we can store (for INT8 signed: -128 and 127).

To use the number again, we do **dequantization**:

$$\hat{x} = s(q - z) \quad (5)$$

The number we get back, $\hat{x}$, is not exactly the same as x . The error is at most half of one step:

$$|x - \hat{x}| \leq \frac{s}{2} \quad (6)$$

2.2 Symmetric Quantization

A simpler version used a lot for weights sets $z = 0$ and uses the same range on both sides of zero:

$$s = \frac{\max |x|}{2^{b-1} - 1}, \quad q = \text{round}\left(\frac{x}{s}\right) \quad (7)$$

2.3 Example by Hand

Suppose the biggest weight is $\max |x| = 2.54$ and we use 8 bits. Then $2^7 - 1 = 127$, so

$$s = \frac{2.54}{127} = 0.02 \quad (8)$$

A weight $x = 1.13$ becomes $q = \text{round}(1.13/0.02) = 57$. When we dequantize: $\hat{x} = 57 \times 0.02 = 1.14$. The error is only 0.01, and 1.14 now fits in 1 byte instead of 4.

2.4 Why Fewer Bits Means More Error

When we use fewer bits, we have fewer integer levels, so the step size s gets bigger and the error gets bigger. Figure 1 shows the step size (when $\max |x| = 1$) for different numbers of bits.

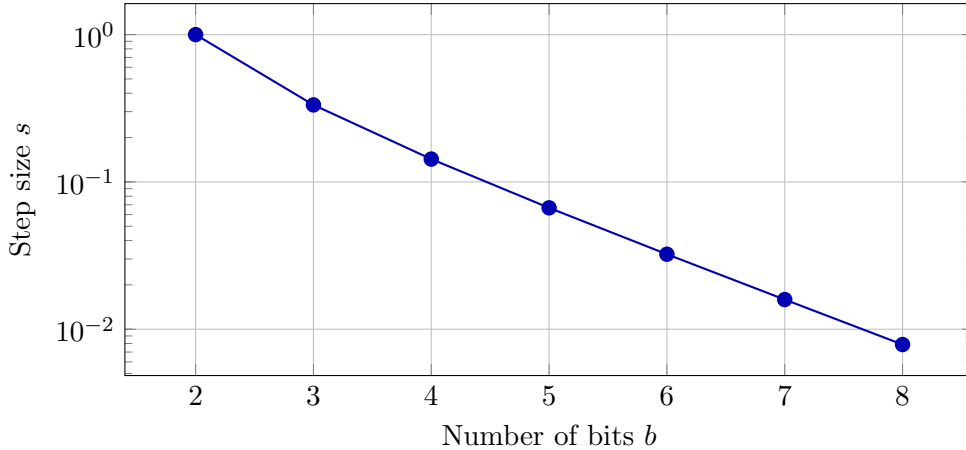


Figure 1: Step size $s = 1/(2^{b-1} - 1)$ for symmetric quantization. Fewer bits means bigger steps and more error.

3 Types of Quantization

- **Post-Training Quantization (PTQ):** we take a model that is already trained and quantize it. It is fast and easy, and needs little or no extra data.
- **Quantization-Aware Training (QAT):** we simulate quantization while training, so the model learns to work well with the rounding error. It gives better accuracy but needs more training time.

- **Dynamic quantization:** the weights are quantized in advance, and the activations (the values that flow through the model) are quantized while the model runs.
- **Static quantization:** both weights and activations are quantized in advance, using some sample data to find their ranges (called calibration).

3.1 Popular Methods for Large Language Models

- **LLM.int8():** 8-bit quantization that keeps the few very large “outlier” values in higher precision.
- **GPTQ:** a post-training method that quantizes weights to 4 bits or less, layer by layer, trying to keep the output the same.
- **AWQ:** protects the most important weights so accuracy stays high.
- **NF4 (used in QLoRA):** a 4-bit data type made for weights that follow a normal (bell-shaped) distribution.

4 Code Examples

4.1 Example 1: Quantization from Scratch

This example quantizes random weights with different numbers of bits and measures the error.

```

1 import numpy as np
2
3 def quantize(x, bits=8):
4     qmax = 2**(bits - 1) - 1
5     s = np.abs(x).max() / qmax
6     q = np.clip(np.round(x / s), -qmax, qmax).astype(np.int8)
7     return q, s
8
9 def dequantize(q, s):
10     return q.astype(np.float32) * s
11
12 weights = np.random.randn(10000).astype(np.float32)
13
14 for bits in [8, 4, 2]:
15     q, s = quantize(weights, bits)
16     err = np.abs(weights - dequantize(q, s)).mean()
17     print(f"{bits}-bit -> mean absolute error: {err:.5f}")

```

The error should get bigger as the number of bits goes down.

4.2 Example 2: Quantizing a Real BERT Model

PyTorch can quantize the Linear layers of BERT to INT8 in one line.

```

1 import os, torch
2 from transformers import AutoModelForSequenceClassification
3
4 model = AutoModelForSequenceClassification.from_pretrained(
5     "bert-base-uncased").eval()
6
7 qmodel = torch.ao.quantization.quantize_dynamic(
8     model, [torch.nn.Linear], dtype=torch.qint8)
9
10 def size_mb(m):
11     torch.save(m.state_dict(), "tmp.pt")
12     size = os.path.getsize("tmp.pt") / 1e6

```

```

13     os.remove("tmp.pt")
14     return size
15
16 print(f"FP32 BERT: {size_mb(model):.0f} MB")
17 print(f"INT8 BERT: {size_mb(qmodel):.0f} MB")

```

The quantized model is usually about 2 times smaller. It is not a full 4 times smaller because only the Linear layers are quantized, and the embedding layer stays in FP32.

4.3 Example 3: Loading LLaMA in 4-bit

With the `bitsandbytes` library, we can load a big model directly in 4-bit. This needs a GPU, and the LLaMA model needs access approval on Hugging Face.

```

1 import torch
2 from transformers import AutoModelForCausalLM, BitsAndBytesConfig
3
4 bnb_config = BitsAndBytesConfig(
5     load_in_4bit=True,
6     bnb_4bit_quant_type="nf4",
7     bnb_4bit_compute_dtype=torch.float16,
8     bnb_4bit_use_double_quant=True,
9 )
10
11 model = AutoModelForCausalLM.from_pretrained(
12     "meta-llama/Llama-2-7b-hf",
13     quantization_config=bnb_config,
14     device_map="auto",
15 )
16 print(f"Memory: {model.get_memory_footprint() / 1e9:.1f} GB")

```

5 Results: Memory by Precision

Figure 2 compares the memory needed for LLaMA-7B at different precisions. Going from FP32 to INT4 makes the model 8 times smaller.

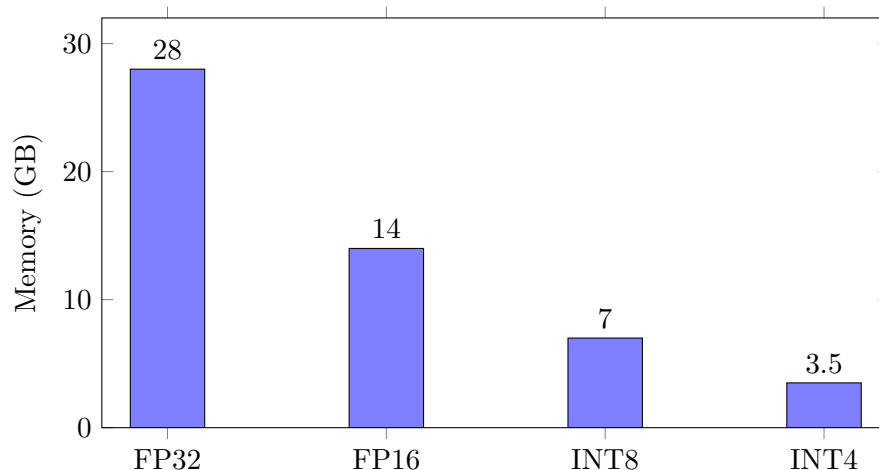


Figure 2: Memory needed for LLaMA-7B weights at different precisions.

6 Advantages and Trade-offs

6.1 Advantages

- **Smaller model:** up to 8 times smaller when going from FP32 to INT4.
- **Less memory:** big models can fit on smaller GPUs.
- **Faster:** smaller numbers move faster through memory, and many CPUs and GPUs are quick with INT8.
- **Cheaper:** less hardware and less energy are needed.
- **Easy to apply:** post-training quantization does not need retraining.

6.2 Trade-offs

- **Accuracy loss:** small at 8-bit, but can be bigger at 4-bit or lower.
- **Outliers:** a few very large values can make the step size too big and hurt accuracy.
- **Hardware support:** the speed-up depends on the hardware and the library we use.

7 Other Ways to Make Models Smaller

Quantization is not the only solution. We can also use:

- **Knowledge distillation:** train a small “student” model to copy a big “teacher” model. For example, DistilBERT is about 40% smaller than BERT.
- **Pruning:** remove weights or neurons that are not important.
- **LoRA:** fine-tune only a few small extra matrices instead of the whole model. This saves memory during training.

These methods can be combined. For example, **QLoRA** uses a 4-bit quantized model together with LoRA, so we can fine-tune large models on one GPU.

8 Conclusion

Large models like BERT and LLaMA need a lot of memory because they store billions of numbers with high precision. Quantization solves this by storing the numbers with fewer bits, such as 8 or 4, which makes the model much smaller and faster. The cost is a small rounding error, which we can control. In practice, quantization is usually the first and simplest technique to try, and together with LoRA it lets us work with large models on limited hardware.